ORBIT.CyberGraph-Node

Team Handover — Neo4j Layer

What was built · How to use it · What comes next

Team 45 · IIIT-H · Schema v1.1.0 · 41/41 Tests Passing

1. What Was Built

The Neo4j graph layer is now complete. It is the local knowledge store that connects every component of the ORBIT Node — Wazuh telemetry, Presidio privacy scans, ORBIT Core intelligence, and human analyst decisions — into a single queryable graph.

In practical terms: when a threat arrives from ORBIT Core, the graph can immediately answer 'which of our assets are vulnerable to this, and do any of them contain sensitive customer data?' That answer is what drives the HITL prioritisation queue.

What	Status	Where to find it
Graph schema — 12 node types, 12 relationship types	**Complete**	`execution/schema/01_apply_schema.py`
Ingestion scripts — all entity types, idempotent	**Complete**	`execution/ingestion/`
ActionCard lifecycle — 7 states, transition enforcement	**Complete**	`execution/lifecycle/actioncard_lifecycle.py`
Delta computation — change detection, export, ack	**Complete**	`execution/delta/compute_delta.py`
41 automated tests — schema, ingestion, lifecycle, delta, contracts	**Complete**	`execution/tests/`
Manual testing guide — Neo4j Browser queries	**Complete**	`execution/tests/MANUAL_TESTING_GUIDE.md`
Integration contracts — ActionCard, Delta, Subscription	**Complete**	`execution/tests/test_contracts.py`

2. How the Graph Fits Into the Node

The graph sits at the centre of the Node. Every other component reads from or writes to it. Nothing in the Node needs to query Wazuh or Presidio directly for context — they query Neo4j.

Component	Direction	What it writes / reads
Wazuh	→ graph (via ingestion scripts)	Host nodes, Service nodes, Vulnerability nodes, HAS_VULNERABILITY relationships
Microsoft Presidio	→ graph (via ingestion scripts)	DataAsset nodes with sensitivity_score, pii_types, crown_jewel, scan_status
ORBIT Core	→ graph (ActionCards in)	ActionCard nodes, Threat nodes, AFFECTS relationships
ORBIT Core	← graph (delta out)	DeltaEvent exports — only changed entities, no raw PII
Node Agent (MCP brain)	reads graph	Crown Jewel assets, active vulnerabilities, pending ActionCards for prioritisation
SOC Analyst (HITL)	→ graph (via lifecycle functions)	ASSIGNED_TO, APPROVED_BY relationships, ActionCard status transitions
NeoDash / Bloom	reads graph	Visualisation of risk, topology, ActionCard status

3. What This Means For Each Team Member

Presidio / Privacy Engine

The graph is ready to receive Presidio output. When your scanner produces a result for a file or database, call ingest_dataasset() with these fields:

```
asset_hash        — HMAC of the file path (never the raw path)
location_pseudonym — pseudonymised path label (e.g. 'db-export-2024-03')
sensitivity_score  — float 0.0-1.0 from Presidio
pii_types          — list of entity types found (e.g. ['EMAIL_ADDRESS', 'CREDIT_CARD'])
host_id            — the host this asset lives on
scan_ts            — ISO8601 timestamp of the scan
```

The graph will automatically compute crown_jewel=True if sensitivity_score >= 0.7. If Presidio is offline, call ingest_dataasset_pending_retry(driver, asset_hash) — it creates a stub node with scan_status='pending_retry' so the delta export can notify Core.

> **Privacy guard is already in place — if location_pseudonym accidentally contains a raw email address or SSN pattern, the script raises PrivacyViolationError before touching the database.**

Wazuh / Telemetry

The graph is ready to receive Wazuh telemetry. For each component:

Wazuh data	Call this function	Minimum required fields

Agent / host inventory	`ingest_host(driver, payload)`	`host_id, hostname, ip, os, agent_version, source='wazuh'`
Running services	`ingest_service(driver, payload)`	`host_id, name, port, proto, source='wazuh'`
Installed packages	`ingest_application(driver, payload)`	`host_id, name, version, vendor, source='wazuh'`
Vulnerability findings	`ingest_vulnerability(driver, payload)`	`host_id, cve_id, cvss, published, summary, severity, source='wazuh'`

All four functions are idempotent — you can call them on every Wazuh poll cycle without creating duplicates. cve_id must be in CVE-YYYY-NNNNN format or the call is rejected.

> **ingest_vulnerability raises OrphanEntityError if the host_id doesn't exist in the graph yet. Always ingest the host before its vulnerabilities.**

DevOps / Infrastructure

The Docker stack is defined in docker-compose.yml. Neo4j runs as orbit-neo4j on ports 7474 (Browser) and 7687 (Bolt). APOC is auto-installed on first start. Key operational commands:

```
# Check health
docker compose ps

# View logs
docker compose logs -f neo4j

# Apply/verify schema (run after any fresh container)
python execution/schema/01_apply_schema.py
python execution/schema/02_verify_schema.py

# Run full test suite
pytest execution/tests/ -v --tb=short
```

For the mock ORBIT Core integration: the ActionCard webhook should POST a Contract A payload to the Node Agent, which calls ingest_actioncard(). The delta push should call compute_delta() + export_delta() and POST the result to Core, then call acknowledge_delta() on success.

> **last_synced_ts is currently in-memory. Before production, wire it to a persistent store — either .env or a metadata node in Neo4j.**

Node Agent / MCP

The graph exposes everything the MCP agent needs for data-aware prioritisation. Key queries to build your tool calls around:

```
// Crown Jewel assets with active vulnerabilities — the core prioritisation query
MATCH (d:DataAsset {crown_jewel: true})-[:RESIDES_ON]->(h:Host)
MATCH (h)-[:HAS_VULNERABILITY]->(v:Vulnerability)
RETURN h.host_id, d.asset_hash, d.sensitivity_score, v.cve_id, v.cvss
ORDER BY d.sensitivity_score DESC, v.cvss DESC

// Pending ActionCards awaiting HITL review
MATCH (ac:ActionCard {status: 'pending'})
OPTIONAL MATCH (ac)-[:AFFECTS]->(target)
RETURN ac.action_id, ac.action_type, ac.confidence, labels(target), target
ORDER BY ac.confidence DESC
```

The full set of 8 visualisation queries is in execution/queries/verification_queries.cypher. Each is annotated with the test case it satisfies.

4. Starting the System

Every session — run these three commands

```
# 1. Start Neo4j (skip if already running)
docker compose up -d

# 2. Activate Python environment
source .venv/bin/activate

# 3. Verify everything is healthy
python execution/schema/00_verify_environment.py
```

Expected output of step 3:

```
✓  Neo4j connectivity verified.
   Server: Neo4j Kernel 5.18.1 (community)
✓  Neo4j server major version is 5.
✓  APOC loaded — version: 5.18.0
✓  All environment checks passed.
```

First time on a fresh Neo4j container

```
python execution/schema/01_apply_schema.py
python execution/schema/02_verify_schema.py
```

Running tests

```
pytest execution/tests/ -v --tb=short
# Expected: 41 passed, 0 failed
```

Neo4j Browser

Open http://localhost:7474 in your browser. Login: neo4j / (password from .env). Paste queries from execution/tests/MANUAL_TESTING_GUIDE.md for visual verification.

5. What Comes Next

The Neo4j layer (D1–D4) is complete. The following work is needed before the full Node is ready for integration testing.

Priority	Item	What it involves	Who
1 — HIGH	**D5: Priority scoring formula**	Define the mathematical combination of CVSS score and sensitivity_score into a single priority rank. crown_jewel is currently a boolean cutoff — D5 gives it a continuous score. Needed for test 41 (MCP prioritisation).	Anushka + Yashav
1 — HIGH	**Wire Wazuh connector**	Connect ingest_host / ingest_service / ingest_vulnerability to live Wazuh API calls. Scripts	Sharif

		accept Wazuh-shaped payloads already — the connector just needs to call them.	
1 — HIGH	**Wire Presidio connector**	Connect ingest_dataasset to Presidio scan output. Privacy guard is already in place in the script.	Anushka
2 — MED	**Mock Core docker stub**	A docker container that can POST valid Contract A ActionCards and receive Contract B delta exports. Needed for integration tests 30–40.	Ayush
2 — MED	**D6: Ops checklist**	Minimal operational procedures: log rotation, backup, schema migration runbook, monitoring queries.	Yashav
2 — MED	**last_synced_ts persistence**	Currently in-memory. Needs persistent store before multi-session delta testing.	Yashav / Ayush
3 — LOW	**D7: Full documentation package**	Consolidate all docs for final handover to client (Satya Vithala / NJSecure).	All
3 — LOW	**Git setup**	Initialise repository, add .env to .gitignore, push initial commit.	Ayush

6. Integration Testing Sequence

Once the Wazuh and Presidio connectors are wired, run the integration tests from the test plan in this order. The Neo4j layer supports all of them.

Test	Use Case	What to verify	Graph assertion
Test 30	UC-01	ORC ingests assets from live Wazuh	`MATCH (a:Asset) RETURN count(a) equals Wazuh agent count`
Test 32	UC-02	Presidio scan flags PII → crown_jewel	`MATCH (d:DataAsset) RETURN d.crown_jewel, d.sensitivity_score`
Test 33	UC-03	Subscription payload — no raw PII	`Inspect POST body to Core mock; assert no IPs or hostnames`
Test 34	UC-04	ActionCard injected as Wazuh alert	`MATCH (ac:ActionCard {status:'pending'}) RETURN ac`
Test 36	UC-05/06	Analyst approves → status=completed	`MATCH (ac)-[:APPROVED_BY]->() RETURN ac.status`
Test 37	UC-07	Delta contains only new agent	`Delta changes[] has exactly 1 entity matching new agent`
Test 38	UC-08	Completion feedback sent to Core	`DeltaEvent.sent_status='sent'; payload has action_id + success:true`
Test 41	R2 MCP	Crown Jewel gets CRITICAL severity	`MATCH (ac:ActionCard {action_id:...}) RETURN ac.confidence`

7. Quick Reference

Key file locations

What you need	Where it is
Start Neo4j + verify	`docker compose up -d then python execution/schema/00_verify_environment.py`
Apply schema to fresh DB	`python execution/schema/01_apply_schema.py`
Run all tests	`pytest execution/tests/ -v --tb=short`
Neo4j Browser	`http://localhost:7474 (login: neo4j / password from .env)`
Manual verification queries	`execution/tests/MANUAL_TESTING_GUIDE.md`
Schema health checks	`execution/tests/SCHEMA_HEALTH_CHECKS.md`
Visualisation queries (Bloom)	`execution/queries/verification_queries.cypher`
All ingestion functions	`execution/ingestion/`
ActionCard lifecycle functions	`execution/lifecycle/actioncard_lifecycle.py`
Delta export / compute	`execution/delta/compute_delta.py`
Custom exceptions	`execution/exceptions.py`
Authoritative spec	`resources/Neo4j_Conceptual_Doc_v1.1_Amended.docx`

Three rules that must never be broken

1. **crown_jewel is always computed from sensitivity_score. Never pass it as input.**
2. **last_updated must be set on every write. Missing it breaks delta exports.**
3. **No raw PII enters Neo4j. location_pseudonym must be hashed/pseudonymised before calling ingest_dataasset.**